

Java – Szybkie Przypomnienie

1. Typy danych i zmienne

Java rozróżnia typy prymitywne (przechowywane na stosie) od obiektowych (przechowywane na stercie). Kluczowa różnica: obiekty mogą być null, prymitywy nie.

Typ	Rozmiar	Zakres	Przykład
byte	8 bit	-128 do 127	byte b = 100;
short	16 bit	-32 768 do 32 767	short s = 30000;
int	32 bit	-2 mld do 2 mld	int x = 42;
long	64 bit	bardzo duży zakres	long l = 123L;
float	32 bit	6-7 cyfr znaczących	float f = 3.14f;
double	64 bit	15-16 cyfr znaczących	double d = 3.14159;
char	16 bit	jeden znak Unicode	char c = 'A';
boolean	1 bit	true / false	boolean ok = true;

UWAGA

String NIE jest typem prymitywnym – to obiekt klasy `java.lang.String`. Porównuj Stringi przez `.equals()`, nie przez `==`.

```
String a = "hello";
String b = "hello";
a == b      // false! (porównuje referencje)
a.equals(b) // true  (porównuje zawartość)
```

1.1 Tablice i var

Tablice mają stały rozmiar. Do dynamicznych kolekcji używaj List (patrz sekcja 6).

```
int[] liczby = {1, 2, 3, 4, 5};
String[] imiona = new String[3];
imiona[0] = "Anna";

var x = 42;    // od Java 10 - typ wnioskowany przez kompilator
var lista = new ArrayList<String>(); // var działa tylko dla zmiennych
lokalnych
```

2. Metody

Metoda to nazwany blok kodu. Może przyjmować parametry i zwracać wartość. Jeżeli nic nie zwraca, typ to void.

```
modyfikator typ nazwaMetody(parametry) { ... } // ogólna składnia

public int dodaj(int a, int b) {
    return a + b;
}

public void wypiszPowitanie(String imie) { // void = brak wartości
    zwracanej
    System.out.println("Czesc, " + imie + "!");
}
```

2.1 Przeciążanie metod (Overloading)

Ta sama nazwa metody, różne listy parametrów. Kompilator wybiera odpowiednią wersję na podstawie argumentów.

```
public int suma(int a, int b) { return a + b; }
public double suma(double a, double b) { return a + b; }
public int suma(int a, int b, int c) { return a + b + c; }
```

2.2 Parametry varargs

Pozwalają przekazać dowolną liczbę argumentów tego samego typu. Wewnątrz metody są traktowane jak tablica.

```
public int sumaWielu(int... liczby) {
    int wynik = 0;
    for (int n : liczby) wynik += n;
    return wynik;
}

sumaWielu(1, 2, 3); // 6
sumaWielu(10, 20, 30, 40); // 100
```

3. Klasy i obiekty

Klasa to szablon (przepis). Obiekt to konkretny egzemplarz stworzony na podstawie tego szablonu za pomocą new.

```
public class Samochod {

    // pola (atrybuty)
    private String marka;
    private int rokProdukcji;
    private double predkosc;

    // konstruktor
    public Samochod(String marka, int rokProdukcji) {
        this.marka = marka;
        this.rokProdukcji = rokProdukcji;
        this.predkosc = 0;
    }

    // getter
    public String getMarka() { return marka; }

    // setter
    public void setRokProdukcji(int rok) { this.rokProdukcji = rok; }

    // metoda biznesowa
    public void przyspieszaj(double delta) {
        predkosc += delta;
    }

    @Override
    public String toString() {
        return marka + " (" + rokProdukcji + ") @ " + predkosc + " km/h";
    }
}
```

Tworzenie i używanie obiektu:

```
Samochod auto = new Samochod("Toyota", 2020);
auto.przyspieszaj(60.0);
System.out.println(auto); // Toyota (2020) @ 60.0 km/h
```

3.1 Modyfikatory dostępu

Modyfikator	Ta sama klasa	Ten sam pakiet	Podklasa	Wszędzie
public	tak	tak	tak	tak
protected	tak	tak	tak	nie
(brak)	tak	tak	nie	nie
private	tak	nie	nie	nie

4. Dziedziczenie

Dziedziczenie pozwala podklasie przejąć pola i metody klasy nadrzędnej. Używaj extends. Java wspiera tylko jednokrotne dziedziczenie klas.

```
// Klasa bazowa (nadrzędna / rodzic)
public class Pojazd {
    protected String nazwa;
    protected int predkoscMaks;

    public Pojazd(String nazwa, int predkoscMaks) {
        this.nazwa = nazwa;
        this.predkoscMaks = predkoscMaks;
    }

    public void jedz() {
        System.out.println(nazwa + " jedzie!");
    }
}

// Klasa pochodna (podklasa / dziecko)
public class Motocykl extends Pojazd {
    private boolean maSidecar;

    public Motocykl(String nazwa, int predkoscMaks, boolean maSidecar) {
        super(nazwa, predkoscMaks); // wywołanie konstruktora rodzica -
MUSI być pierwsza linia
        this.maSidecar = maSidecar;
    }

    @Override // nadpisanie metody rodzica
    public void jedz() {
        super.jedz(); // można wywołać oryginalną wersję przez super
        System.out.println("Vroom! Sidecar: " + maSidecar);
    }
}
```

UWAGA

@Override to adnotacja – nie jest obowiązkowa, ale chronisz się przed literówkami. Kompilator zgłosi błąd, jeśli metoda nie istnieje w klasie nadrzędnej.

4.1 final i abstract

Modyfikator final na klasie uniemożliwia dziedziczenie. Modyfikator abstract wymaga dziedziczenia i implementacji oznaczonych metod.

```
public final class String { ... } // nie można jej rozszerzać

public abstract class Kształt { // nie można utworzyć obiektu Kształt
    public abstract double polePowierzchni(); // brak ciała - podklasa
MUSI zaimplementować
    public void wypisz() {
        System.out.println("Pole: " + polePowierzchni());
    }
}

public class Kolo extends Kształt {
    private double promien;
    public Kolo(double r) { this.promien = r; }
```

```
@Override
public double polePowierzchni() { return Math.PI * promien * promien;
}
}
```

4.2 Polimorfizm

Obiekt podklasy może być traktowany jako obiekt klasy nadrzędnej. Metoda wywołana zależy od rzeczywistego typu obiektu (late binding).

```
Kształt k1 = new Kolo(5);
Kształt k2 = new Prostokat(4, 6);

k1.wypisz(); // Pole: 78.54... // wywoła Kolo.polePowierzchni()
k2.wypisz(); // Pole: 24.0    // wywoła Prostokat.polePowierzchni()

// instanceof - sprawdzenie prawdziwego typu
if (k1 instanceof Kolo kolo) { // pattern matching (Java 16+)
    System.out.println(kolo.promien);
}
```

5. Kolekcje

Pakiet java.util. Zawsze używaj typów generycznych (<T>) zamiast surowego typu.

5.1 List – lista (z duplikatami, z kolejnością)

```
List<String> lista = new ArrayList<>();
lista.add("Anna");
lista.add("Bartek");
lista.add("Anna");           // duplikat OK
lista.get(0);                // "Anna"
lista.size();                // 3
lista.remove("Bartek");
lista.contains("Anna");      // true

// iteracja - for-each (zalecane)
for (String imie : lista) {
    System.out.println(imie);
}

// iteracja - Stream + lambda (nowoczesne)
lista.forEach(imie -> System.out.println(imie));
```

5.2 Set – zbiór (bez duplikatów, bez gwarantowanej kolejności)

```
Set<Integer> zbior = new HashSet<>();
zbior.add(1);
zbior.add(2);
zbior.add(1); // ignorowane - duplikat!
zbior.size(); // 2

LinkedHashSet - zachowuje kolejnosc wstawiania
TreeSet       - sortuje elementy naturalnie
```

5.3 Map – mapa klucz-wartosc

```
Map<String, Integer> wiek = new HashMap<>();
wiek.put("Anna", 25);
wiek.put("Bartek", 30);
wiek.get("Anna");           // 25
wiek.getDefault("Celina", 0); // 0 - brak klucza
wiek.containsKey("Bartek"); // true
wiek.remove("Bartek");

for (Map.Entry<String, Integer> e : wiek.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

5.4 Porównanie kolekcji

Kolekcja	Duplikaty	Kolejność	Null	Kiedy używać
ArrayList	tak	indeks	tak	najczęściej używana
LinkedList	tak	wstawiania	tak	duzo wstawien/usuniec
HashSet	nie	brak	1x	szybkie sprawdzanie

Kolekcja	Duplikaty	Kolejność	Null	Kiedy używać
TreeSet	nie	sortowana	nie	posortowany zbiór
HashMap	kl.nie	brak	1x	szybki dostęp po kl.
TreeMap	kl.nie	kluczy	nie	posortowane klucze
